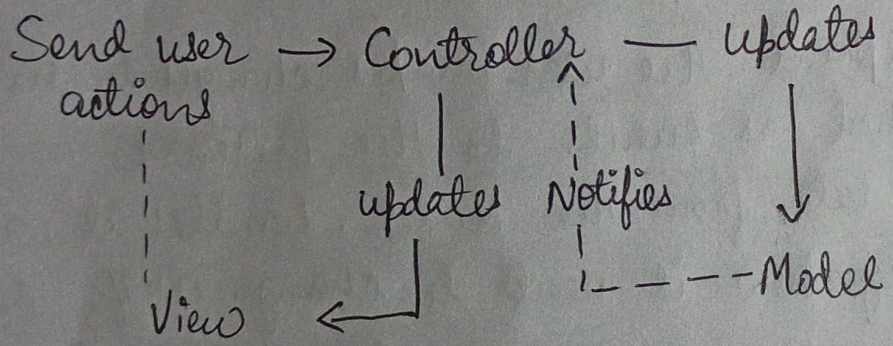
Name - Nandita Singh
Regd No - 21411019333

Assignment - 1 : Design Pattern Explanation

→ What is MVC ?

The Model View controller (MVC) framework is an architectural design pattern that separates an application into three main logical components Model, View and controller. Each architectural component is built to handle specific development aspects of an application. It was traditionally used for desktop graphical user interfaces (GUIs).

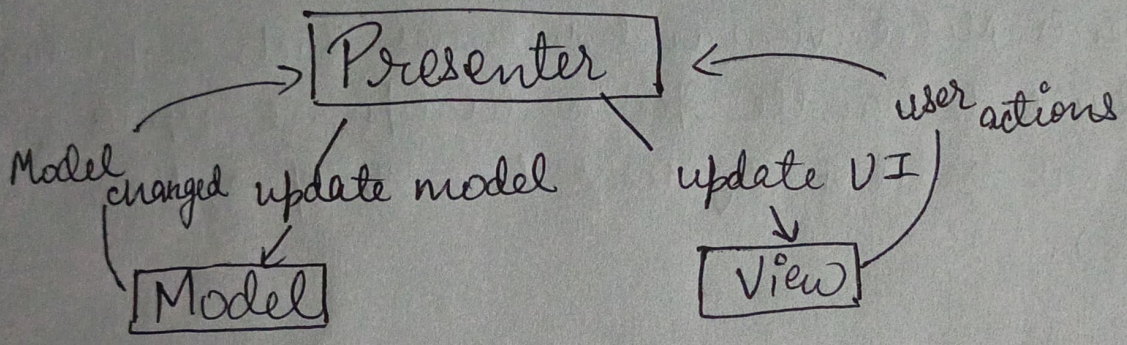


Structurally, the controller accepts user input and updates the model. The model processes information and informs the view if there is any update. The view fetches updated information from the Model and updates the UI. This structure enables a clean distinction between how the application functions (Model) how it is governed (Controller) and how it appears (View). MVC is widely used in web frameworks such as Django, ASP.NET MVC etc.

Variants of MVC

1) MVP (Model-View-Presenter)

The Model-View-Presenter pattern is a MVC variant in which the presenter acts more actively. In MVP, the view becomes more passive and relies entirely upon the Presenter for logic and the updating the UI. The Presenter gets input from the View, communicates with the Model to recieve or update information and sends the results to the view for updating.

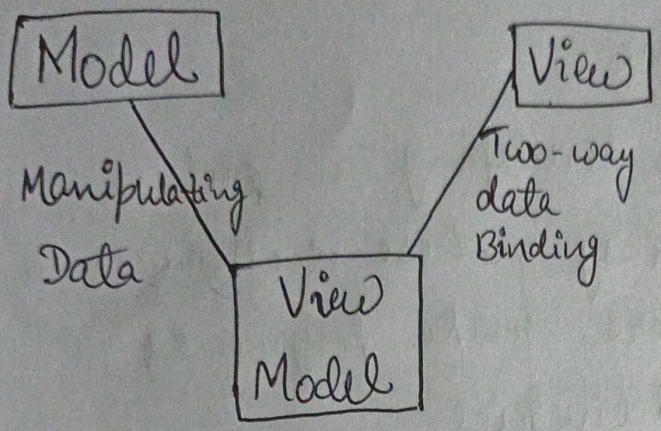


From a structural standpoint, the user makes changes to the view and it reports them as events to the Presenter.
The presenter requests data from the Model, handles it, and informs the view precisely how it needs to change itself.
MVP works well with apps that require UI logic to be separated to allow testing or enhanced control such as for desktop applications such as windows form.

2) MVVM (Model View ViewModel) -

The Model-View-ViewModel pattern is a derivative of MVC common in contemporary UI frameworks. With MVVM, the ViewModel mediates between view and model. In contrast with MVP, however, the View does not exchange messages directly with the ViewModel, but rather works through data

binding. This indicates that when changes occur in data within the viewModel, the corresponding View is likewise updated, and vice-versa.



The framework facilitates a two-way binding between view and viewModel. This minimizes the amount of code to be written in the View and makes it more maintainable. MVVM is particularly beneficial in frameworks that provide out of the box support for data binding, including Angular or Android. It is optimum when you wish to have a clean separation between UI logic and UI components and when data binding can make it easier to interact between the View and viewModel.